UNDERSTANDING FINANCIAL MARKET DYNAMICS

Comprehensive Report

CAPSTONE PROJECT SEMESTER 01

prepared by :

ARCHIT
GUPTA [redacted]

INTRODUCTION TO THE PROJECT

As a young data analyst at FinScoop—a fictional Indian financial services company tasked with understanding how market shocks can affect major financial institutions in India, such as SBI, ICICI, HDFC, and Axis Bank.

My goal was to use Python, & Linear Algebra to analyze how shocks in the market affect the stability of these financial institutions. As I worked through this project, I gained practical skills in Python programming and also understood how interconnected financial systems react to market shocks.

STUDENT DETAILS

Name: [redacted]

Roll No.: [redacted]

Institute: Indian Institute of Technology Jodhpur

Program: B.Sc/B.S in Applied AI & Data Science

Department: AIDE

Course Code: AIL 1010

Course Name: Linear Algebra & Numerical Analysis

MODEL INPUTS & PARAMETERS

For my analysis, I used the following matrix A, which represents the interconnectedness between four major financial institutions (e.g., SBI, ICICI, HDFC, Axis Bank). Each entry reflects how much one institution is influenced by another.

$$A = \begin{vmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{vmatrix}$$

This matrix assumes that:

- Each institution is mainly influenced by itself (diagonal=2).
- Each institution is also directly influenced by its neighbour (off-diagonal = -1).
- There's no direct influence from institutions two steps away (zeroes elsewhere).

I used the following vector b representing external market shocks & vector x as the initial portfolio vector:

$$b = \begin{vmatrix} 1 \\ 2 \\ 2 \\ 1 \end{vmatrix} \qquad x = \begin{vmatrix} 1 \\ 1 \\ 0 \\ 0 \end{vmatrix}$$

BASICS OF MATRICES IN FINANCIAL MODELLING

In the financial network, symmetry of matrices is very crucial, because it denotes **reciprocal influence**. This means if Bank A influences Bank B (here represented by -1), does Bank B influence Bank A by the exact same amount or not.

A matrix is symmetric only when the transpose of a matrix equals to the matrix itself.

PYTHON IMPLEMENTATION

```python
import numpy as np
import sympy as sp
import scipy
from sympy import init_printing

#Displaying the original matrix
m1=np.array([[2,-1,0,0],[-1,2,-1,0],[0,-1,2,-1],[0,0,-1,2]])
init_printing(use_latex='mathjax')
print("The matrix below shows the interconnected relationships among the financial institutions: \n")
display(sp.Matrix(m1))

#Checking Matrix Symmetry
m1_transpose=m1.T
if (m1==m1_transpose).all():
    print("\nThe matrix is symmetric")
else:
  print("\nThe matrix is not symmetric")
```

```
The matrix below shows the interconnected relationships among the financial institutions:
```

$$\begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}$$

```
The matrix is symmetric
```

BASICS OF MATRICES IN FINANCIAL MODELLING

Calculation of Rank, Determinant, Trace & Null Space as specific properties of matrix A to analyze **stability of the financial network**.

Stability of the financial network refers to the system's ability to absorb economic shock without collapsing or spiraling out of control.

RANK (Number of linearly independent rows in a matrix)

If the rank is full (4 in this case), the banks act as distinct entities. If the rank is lower, it means the behavior of some banks is entirely dependent on others (redundancy).

Example: Suppose Axis Bank always does exactly what SBI, ICICI does combined. Then in this case the rank of the matrix would be <4. So, if SBI drops 2% and ICICI drops 5%, Axis Bank must drop 7% by following a simple combined effect. Here, we don't have to even need to watch Axis Bank's stock ticker; we already know the answer just by watching the others. It's redundant.

DETERMINANT (Scalar value derived from a matrix)

If the determinant is non-zero, there is a specific predictable reaction to a shock. If it is non-zero, the system is singular/unstable.

BASICS OF MATRICES IN FINANCIAL MODELLING

TRACE (The sum of values on the main diagonal)

This represents the aggregate **self influence** of the system. It measures the total amount of internal stability present in the entire banking market comprising the given banks.

It's a summation of all the banks in terms of how much each of them relies on it's own internal capital, history, and policies, rather than just reacting to other situations arising in the market.

NULL SPACE (The number of zero columns/rows of a matrix in Reduced Row Echelon Form)

Scenario A: Empty Null Space
This is regarded as the best case, because this means the system is transparent. If SBI or Axis Bank makes a move, the market will see it. Nothing hidden.

Scenario B: Non-Empty Null Space
This case is regarded as risky! Enormous changes might be happening inside these banks, but the dashboard shows everything is calm. The risk is invisible.

BASICS OF MATRICES IN FINANCIAL MODELLING
PYTHON IMPLEMENTATION

```
#Calculating Matrix Rank
m1_rank=np.linalg.matrix_rank(m1)
print(f"\nThe rank of matrix is: {m1_rank}")

#Calculating Matrix Determinant
m1_determinant=np.linalg.det(m1)
print(f"\nThe determinant of matrix is: {m1_determinant }")

#Calculating Matrix Trace
m1_trace=np.trace(m1)
print(f"\nThe trace of matrix is: {m1_trace}")

#Calculating Matrix Null Space
m1_nullspace=scipy.linalg.null_space(m1)
print(f"\nThe null space of matrix is: {m1_nullspace}")
```

```
The rank of matrix is: 4

The determinant of matrix is: 4.999999999999999

The trace of matrix is: 8

The null space of matrix is: []
```

BASICS OF MATRICES IN FINANCIAL MODELLING

SOLVING THE *A x = B* EQUATION

***A* (The System):** The interconnected banking rules (who affects whom).

***B* (The Shock):** The pressure applied to the market (e.g., inflation, regulation change).

***x* (The Unknown):** The set of values that we calculate from the given relation in order to understand how portfolio or the banks' value must adjust to balance out that external shock.

PYTHON IMPLEMENTATION

```python
#Checking for solution of x using Ax=B
A=np.array([[2,-1,0,0],[-1,2,-1,0],[0,-1,2,-1],[0,0,-1,2]])
init_printing(use_latex='mathjax')
print(f"\nThe Matrix A is: \n")
display(sp.Matrix(A))

B=np.array([[1],[2],[2],[1]])
init_printing(use_latex='mathjax')
print(f"\nThe Matrix B is: \n")
display(sp.Matrix(B))

x=np.linalg.solve(A,B)
print(f"\nThe solution of x is: \n")
display(sp.Matrix(x))
```

```
The Matrix A is:
```

$$\begin{bmatrix} 2 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 2 \end{bmatrix}$$

```
The Matrix B is:
```

$$\begin{bmatrix} 1 \\ 2 \\ 2 \\ 1 \end{bmatrix}$$

```
The solution of x is:
```

$$\begin{bmatrix} 3.0 \\ 5.0 \\ 5.0 \\ 3.0 \end{bmatrix}$$

FINANCIAL NETWORK CONTAGION

Using Eigenvectors and Eigenvalues, we shall now explore how market shocks propagate through the interconnected financial network.

EIGENVECTORS

Markets usually crash in specific patterns. An Eigenvector represents a Fundamental Market Scenario that the network naturally wants to move in. These are the specific patterns of behavior our network is built to experience.

Example: All the banks might just drop together representing a total market collapse. Another case might be of one rising and the other dropping representing a competitive shift. There can be many such cases depending on the nature and type of market we are dealing with.

EIGENVALUES (λ)

Eigenvalue is the number attached to that scenario. It tells you **how intense** a specific scenario becomes over time.

If $\lambda > 1$ (Growth): If a shock hits in this pattern, it will grow. A small problem becomes a massive crisis. The network amplifies the stress.

If $\lambda < 1$ (Decay): If a shock hits in this pattern, it will fade away. The network absorbs the stress and calms down.

If $\lambda = 0$: The shock disappears instantly.

FINANCIAL NETWORK CONTAGION

THE LARGEST EIGENVALUE: "THE EXPLOSION RISK"

This number measures the maximum amplification of the system.

Stability Indicator:

- **High Value:** The banks are reactive and volatile. A single event (like a regulation change) causes massive price swings.
- **Low/Moderate Value:** The banks are calm. They absorb the shock without swinging wildly.

THE SMALLEST EIGENVALUE: "THE SENSITIVITY RISK"

This number measures the minimum resistance of the system.

Stability Indicator:

- **Tiny Value (Near 0):** The system is "Vulnerable." A tiny, almost invisible error in the market data can result in a massive error in your predicted bank prices.
- **Larger Value:** The system is robust. It resists errors.

FINANCIAL NETWORK CONTAGION

To judge if the whole network is stable, analysts divide the two:

$$\text{Condition Number} = \frac{\text{Largest Eigenvalue}}{\text{Smallest Eigenvalue}}$$

- **Small Ratio (e.g., 1 to 10):** STABLE

The gap between the strongest and weakest response is small. The banks behave predictably.

- **Huge Ratio (e.g., 1000+):** UNSTABLE

The system is unpredictable. It reacts wildly differently depending on exactly which angle the shock comes from.

VALIDATION USING CAYLEY-HAMILTON THEOREM

Every matrix has a unique **Characteristic Equation**. It states that if the matrix is plugged into this equation, it should result in 0 always.

$$\det(A - \lambda I) = 0$$

If this equation does not result in 0, then there is something wrong with the matrix definition or calculations.

FINANCIAL NETWORK CONTAGION

PYTHON IMPLEMENTATION

```
#Calculating Eigenvalues, and Eigenvectors
eigval,eigvec=np.linalg.eig(m1)
print(f"The Eigenvalues of matrix are:\n")
display(sp.Matrix(eigval))
print(f"\nThe Eigenvectors of matrix are:\n")
display(sp.Matrix(eigvec))

print(f"\nThe Dominant Eigenvalue is: {eigval[0]}")
print(f"\nThe Vulnerable Eigenvalue is: {eigval[2]}")

#Validating Cayley-Hamilton Theorem
I=np.eye(4) #Identity Matrix

#Finding the Equation
"""np.poly(A) automatically finds the characteristic equation coefficients"""
coeffs=np.poly(A)
print("\nThe Equation numbers:", coeffs)

#Plug A into the Equation
"""Calculate powers using '@'"""
A2=A@A        # A squared
A3=A@A2       # A cubed
A4=A@A3       # A to the power of 4

#The Formula: c0*A^4 + c1*A^3 + c2*A^2 + c3*A + c4*I
result=(coeffs[0] * A4 +
        coeffs[1] * A3 +
        coeffs[2] * A2 +
        coeffs[3] * A  +
        coeffs[4] * I)

#Final Check
"""Does the result equal Zero? (Checks if numbers are extremely close to 0)"""
print("\nIs the theorem valid?", np.allclose(result, 0))

if np.allclose(result, 0):
  print("="*40)
  print("SUCCESS: The matrix obeys its own equation.")
else:
  print("FAIL: Something is wrong.")
```

FINANCIAL NETWORK CONTAGION

PYTHON IMPLEMENTATION

```
The Eigenvalues of matrix are:
```

$$\begin{bmatrix} 3.61803398874989 \\ 2.6180339887499 \\ 0.381966011250105 \\ 1.38196601125011 \end{bmatrix}$$

```
The Eigenvectors of matrix are:
```

$$\begin{bmatrix} -0.371748034460184 & -0.601500955007546 & -0.371748034460185 & -0.601500955007546 \\ 0.601500955007545 & 0.371748034460185 & -0.601500955007546 & -0.371748034460184 \\ -0.601500955007546 & 0.371748034460184 & -0.601500955007546 & 0.371748034460185 \\ 0.371748034460185 & -0.601500955007546 & -0.371748034460185 & 0.601500955007546 \end{bmatrix}$$

```
The Dominant Eigenvalue is: 3.6180339887498913

The Vulnerable Eigenvalue is: 0.3819660112501053

The Equation numbers: [  1.  -8.  21. -20.   5.]

Is the theorem valid? True
========================================
SUCCESS: The matrix obeys its own equation.
```

PORTFOLIO RISK ASSESSMENT

Checking whether the our portfolio lies within the span of the dominant eigenvectors of the 2 largest eigenvalues, helps us identify whether our portfolio is exposed to potential risk or not.

Likewise, simulating how the stock prices of SBI, Axis Bank, HDFC, and ICICI would evolve under a market shock, based on our initial linear model gives us a practical overview of the whole situation. This would help in creating reasonable solutions for stabilizing the financial network during economic turbulence.

PYTHON IMPLEMENTATION

```
# Define the Portfolio Vector X
X = np.array([[1],[1],[0],[0]])

#Compute Eigenvalues and Eigenvectors
eigenvalues, eigenvectors = np.linalg.eig(A)
print(f"The Eigenvalues of matrix are:\n")
display(sp.Matrix(eigval))
print(f"\nThe Eigenvectors of matrix are:\n")
display(sp.Matrix(eigvec))

#Find the two largest (dominant) eigenvalues and their vectors
#Get indices that have sorted the eigenvalues from large to small
sorted_ir
"""In Nun
Instead c
print(sor

#Select t
top_2_inc

#Extract
v1=eigenv
v2=eigenv
"""The ei
gives you
```

```
#Check if X lies in the span of v1 and v2
"""We form a matrix with v1, v2, and X.
If the Rank of [v1, v2] is equal to the Rank of [v1, v2, X],
then X adds no new dimension and lies within the span."""

matrix_span=np.column_stack((v1, v2))
matrix_with_x=np.column_stack((v1, v2, X))

rank_span=np.linalg.matrix_rank(matrix_span)
rank_with_x=np.linalg.matrix_rank(matrix_with_x)

print(f"\nRank of dominant eigenvectors: {rank_span}")
print(f"\nRank including portfolio X: {rank_with_x}")

if rank_span == rank_with_x:
    print("\nResult: Yes, the portfolio lies within the span.")
else:
    print("\nResult: No, the portfolio does NOT lie within the span.")
```

PORTFOLIO RISK ASSESSMENT

PYTHON IMPLEMENTATION

```python
#Check if X lies in the span of v1 and v2
"""We form a matrix with v1, v2, and X.
If the Rank of [v1, v2] is equal to the Rank of [v1, v2, X],
then X adds no new dimension and lies within the span."""

matrix_span=np.column_stack((v1, v2))
matrix_with_x=np.column_stack((v1, v2, X))

rank_span=np.linalg.matrix_rank(matrix_span)
rank_with_x=np.linalg.matrix_rank(matrix_with_x)

print(f"\nRank of dominant eigenvectors: {rank_span}")
print(f"\nRank including portfolio X: {rank_with_x}")

if rank_span == rank_with_x:
    print("\nResult: Yes, the portfolio lies within the span.")
else:
    print("\nResult: No, the portfolio does NOT lie within the span.")
```

```python
import pandas as pd
import matplotlib.pyplot as plt

#Define Vector b (External Shocks)
b = np.array([[1],[2],[2],[1]])

#Define Vector X (Initial Stock Prices)
X = np.array([[1],[1],[0],[0]])

#Run the Simulation
steps = 10          # Number of days/weeks to simulate
history = [X]       # List to store price history
current_x = X

print("\nStarting Simulation...")
for t in range(steps):
    #Formula: New Price = (Matrix * Old Price) + Shock
    next_x = A.dot(current_x) + b

    # Store and update
    history.append(next_x)
    current_x = next_x
```

PORTFOLIO RISK ASSESSMENT

PYTHON IMPLEMENTATION

```
#Create Dataframe to view results
df = pd.DataFrame(np.array(history).squeeze(), columns=['SBI', 'ICICI', 'HDFC', 'Axis'])
print(df)

#Visualize the Evolution (Graph)
plt.figure(figsize=(10, 6))
plt.plot(df.index, df['SBI'], marker='o', label='SBI')
plt.plot(df.index, df['ICICI'], marker='o', label='ICICI')
plt.plot(df.index, df['HDFC'], marker='o', label='HDFC')
plt.plot(df.index, df['Axis'], marker='o', label='Axis')

plt.title('Stock Price Evolution Under Market Shock')
plt.xlabel('Time Steps')
plt.ylabel('Stock Price')
plt.legend()
plt.grid(True, linestyle='--')
plt.savefig('stock_evolution.png') # Saves the graph as an image
plt.show()
```

```
The Eigenvalues of matrix are:
```

$$\begin{bmatrix} 3.61803398874989 \\ 2.6180339887499 \\ 0.381966011250105 \\ 1.38196601125011 \end{bmatrix}$$

```
The Eigenvectors of matrix are:
```

$$\begin{bmatrix} -0.371748034460184 & -0.601500955007546 & -0.371748034460185 & -0.601500955007546 \\ 0.601500955007545 & 0.371748034460185 & -0.601500955007546 & -0.371748034460184 \\ -0.601500955007546 & 0.371748034460184 & -0.601500955007546 & 0.371748034460185 \\ 0.371748034460185 & -0.601500955007546 & -0.371748034460185 & 0.601500955007546 \end{bmatrix}$$

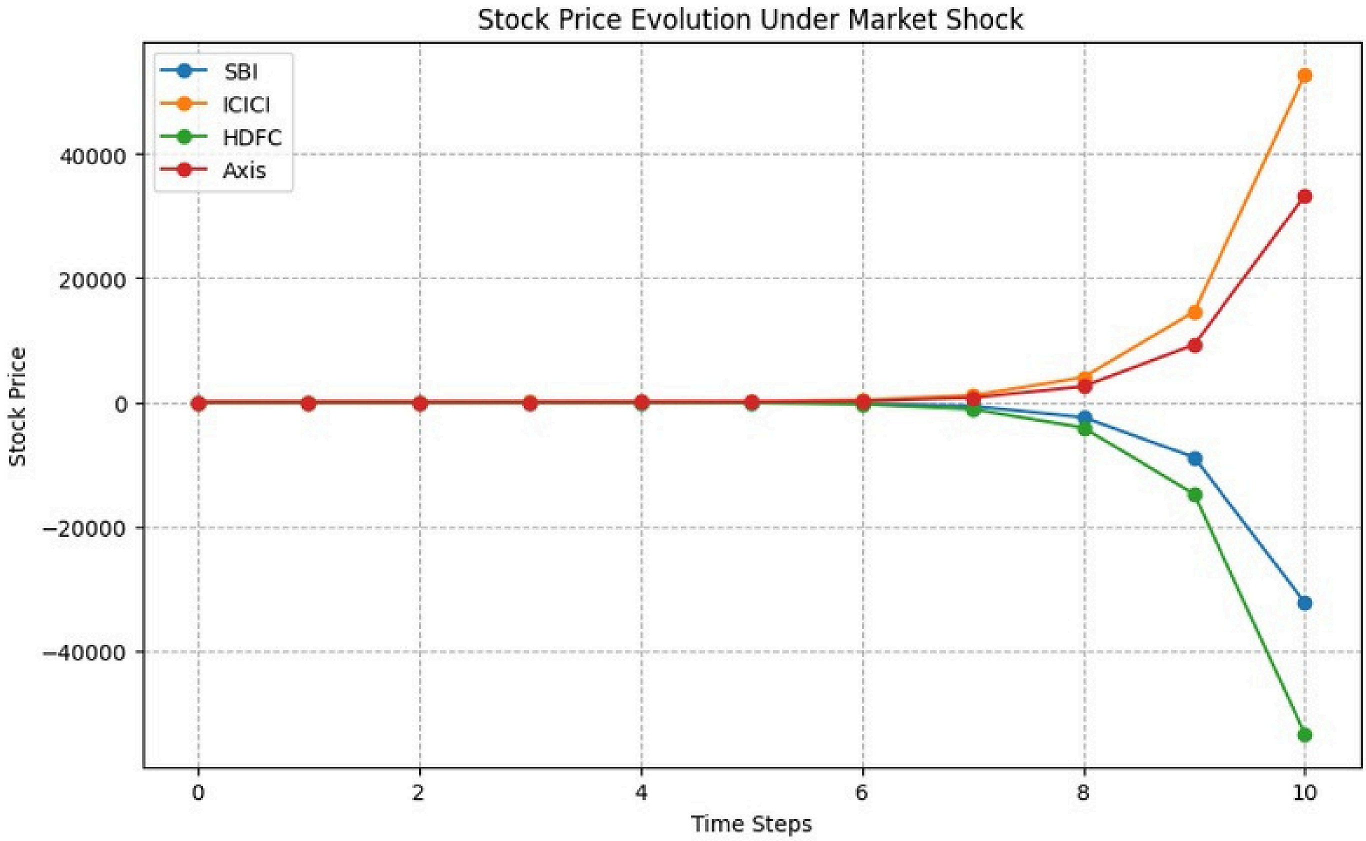
```
Rank of dominant eigenvectors: 2

Rank including portfolio X: 3

Result: No, the portfolio does NOT lie within the span.
```

```
Starting Simulation...
       SBI  ICICI   HDFC   Axis
0        1      1      0      0
1        2      3      1      1
2        2      5      0      2
3        0     10     -5      5
4       -9     27    -23     16
5      -44     88    -87     56
6     -175    309   -316    200
7     -658   1111  -1139    717
8    -2426   4021  -4104   2574
9    -8872  14574 -14801   9253
10  -32317  52823 -53427  33308
```

PORTFOLIO RISK ASSESSMENT
PYTHON IMPLEMENTATION



INTERPRETATION

The numbers are getting very large (exploding) or oscillating widely, depicting instability. The shock does not fade away, but the interconnectedness (A) amplifies them, causing a financial crisis in our simulation.

The -ve values can be interpreted as **bankruptcy** or **total collapse** of that institution.

PORTFOLIO RISK ASSESSMENT

KEY HIGHLIGHTS

1. The financial network satisfies the reciprocal influence.
2. Since the rank is full, all the banks act as distinct entities in the network, and their functioning is not dependent on each other.
3. The determinant is non - zero highlighting that there is a specific predictable reaction to the given shock.
4. The system is transparent, as evident from the empty null space.
5. The system is structurally unstable as the dominant eigenvalue > 1.
6. Our portfolio is unique as the ranks of the Eigenvector is not equal to the rank of the matrix containing the Eigenvector and the portfolio. This emphasizes on the fact that our portfolio is unique. It cannot be explained just by the 2 biggest market forces. It contains risks or factors that are separate from the dominant market trend.
7. The dynamic simulation check confirmed that the network is unstable and verifies the facts gained from the explosion risks' and sensitivity risks' stability indicators.

Conclusion

The simulation proves that the current financial network is fragile to withstand these specific market shocks without any external intervention

PORTFOLIO RISK ASSESSMENT

RECOMMADATIONS

Decoupling This will reduce the dependency between banks so that a crash in one doesn't spread to another. Lowering the -1 to almost 0 will be very beneficial.

Capital Buffers

Making the banks individually stronger will help it absorb shocks without needing any external help. This would mean increasing the **Capital Adequacy Ratio** for all the banks.

Government Intervention

Government intervention is required in order to lower the shock vector entries, especially for the ones that are on higher risk. This would also reduce the effect on the overall network too, reducing the impact on other banks.

DISCLAIMER

1. Fictitious Scenario and Entities

This report is based entirely on a fictitious and hypothetical scenario. The names of institutions, organizations, and financial entities mentioned herein are used fictitiously. Any resemblance to actual events, locales, or persons, living or dead, is entirely coincidental.

2. No Intent of Harm

The contents of this report are illustrative in nature. There is no intention to lower the reputation, defame, or cause financial or reputational harm to any existing financial institutions or market entities. All analyses and critiques presented are strictly theoretical.

3. Academic Purpose

This document has been prepared solely for academic and educational purposes. It should not be construed as professional financial advice, investment guidance, or a real-world market analysis. I assumes no responsibility or liability for any errors or omissions in the content of this site. The information contained in this report is provided on an "as is" basis with no guarantees of completeness, accuracy, or timeliness.

DISCLAIMER

4. Use of Artificial Intelligence

Please note that Artificial Intelligence (AI) tools were utilized extensively during the preparation of this report. AI was employed to assist in understanding concepts related to integrated finance, to generate suitable recommendations, and elsewhere. While I have reviewed the output, the reliance on AI models means the report may contain interpretative limitations.

END OF REPORT